

A
PROJECT REPORT
on
An Explainable and Adaptive Machine Learning
Framework for Phishing Website Detection

for the submission of the
requirement of the degree of

Bachelor of Technology
in
Computer Science and Engineering (AIML)



Submitted To:

Dr. Rajendra Kachhwaha
Professor

Submitted By:

Nitin Ola (23EJIAI082)
Pukhraj (23EJIAI090)

Department of Computer Science and Engineering (AIML)
Jodhpur Institute of Engineering & Technology, Jodhpur
Session 2026-27

Certificate

This is to certify that the Project entitled **An Explainable and Adaptive Machine Learning Framework for Phishing Website Detection** has been carried out by the student/team of college Jodhpur Institute of Engineering and Technology, Jodhpur under my guidance and supervision of **Dr. Rajendra Kachhwaha** in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering (AIML) during the academic year 2026-27.

Team:

1. Nitin Ola (23EJIAI082)
2. Pukhraj (23EJIAI090)

Date: 26 Sept 2026

Place: JIET, Jodhpur

Signature of Guide

Signature of H.O.D

Dr. Rajendra Kachhwaha
Professor

Dr. Pratibha Peshwa
Head of Department

Acknowledgment

I would like to express my deepest gratitude to all those who have contributed to the successful completion of this project. First and foremost, I would like to thank my project guide, **Dr. Rajendra Kachhwaha**, Department of Computer Science and Engineering (AIML), Jodhpur Institute of Engineering and Technology, for his invaluable guidance, continuous support, and encouragement throughout the duration of this project. His expertise and insightful feedback have been instrumental in shaping this work.

I am also grateful to the **Dr. Pratibha Peshwa** and all faculty members of the Department of Computer Science and Engineering (AIML) for providing the necessary infrastructure and academic environment that facilitated this research.

I would like to extend my sincere thanks to my project partner, Nitin Ola, Pukhraj for the collaborative effort, constructive discussions, and shared dedication that made this project possible. The teamwork and mutual support have been truly invaluable.

I am also thankful to my friends and colleagues who provided technical assistance and moral support during challenging phases of this project.

Finally, I would like to express my profound gratitude to my family for their unconditional love, patience, and encouragement throughout my academic journey. Their support has been my foundation and motivation.

Pukhraj (23EJIAI090)
Nitin Ola (23EJIAI082)

Abstract

Phishing attacks have emerged as one of the most significant cybersecurity threats in the modern digital landscape. These attacks exploit human psychology through social engineering and deceptive websites to steal sensitive information such as login credentials, financial data, and personal identification details. The increasing sophistication and rapid evolution of phishing techniques necessitate the development of intelligent, adaptive, and explainable detection systems.

This project presents an **Explainable and Adaptive Machine Learning Framework for Phishing Website Detection** that leverages URL-based characteristics to classify websites as either legitimate or phishing.

The system implements and compares multiple classical machine learning algorithms including Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, Gradient Boosting Classifier, CatBoost, Decision Tree, K-Nearest Neighbors, Multi-layer Perceptron, and Naive Bayes. Additionally, the framework explores deep learning models such as Artificial Neural Networks (ANN) and Long Short-Term Memory (LSTM) networks for enhanced detection capabilities.

A critical contribution of this work is the integration of Explainable Artificial Intelligence (XAI) techniques, specifically SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations), to provide human-understandable explanations for each prediction. This transparency is essential for building trust in automated security systems and enabling security analysts to understand the factors contributing to classification decisions.

The experimental results demonstrate that the Gradient Boosting Classifier achieves the highest accuracy of 97.4%, with an F1-score of 0.977, Recall of 0.994, and Precision of 0.986. The feature importance analysis reveals that URL characteristics such as HTTPS token, Anchor URL, Website Traffic, and URL length are among the most significant predictors of phishing activity.

The framework is deployed as a web application using Flask, providing a simple and intuitive interface where users can submit URLs and receive immediate classification results along with detailed explanations of the contributing features. The modular architecture ensures maintainability and facilitates future extensions toward adaptive learning, real-time crawling, and concept drift detection.

Contents

Certificate	2
Acknowledgment	3
Abstract	4
1 Introduction	10
1.1 Project Overview	10
1.1.1 Problem Statement	10
1.1.2 Project Objectives	11
1.2 Scope of the Project	11
1.3 Technologies Used	12
1.3.1 Programming Languages	12
1.3.2 Databases	12
1.3.3 Frameworks and Libraries	12
2 Literature Survey	13
2.1 Introduction to Phishing Detection	13
2.2 Blacklist-Based Approaches	13
2.3 Heuristic and Rule-Based Approaches	13
2.4 Machine Learning Approaches	14
2.4.1 Classical Machine Learning	14
2.4.2 Ensemble Methods	14
2.5 Deep Learning Approaches	15
2.5.1 Artificial Neural Networks (ANN)	15
2.5.2 Recurrent Neural Networks (RNN/LSTM)	15
2.5.3 Convolutional Neural Networks (CNN)	15
2.6 Explainable AI in Security	15
2.6.1 SHAP (SHapley Additive exPlanations)	15
2.6.2 LIME (Local Interpretable Model-agnostic Explanations)	15
2.7 Research Gap and Motivation	15

2.8	Summary of Literature	16
3	Requirement Specification	17
3.1	General Requirements	17
3.1.1	Functionalities	17
3.1.2	Use Case Model	18
3.1.3	General Constraints	18
3.1.4	Supplementary Requirements	19
3.2	Functional Requirements	20
3.3	Non-Functional Requirements	20
4	Architectural Design	21
4.1	System Architecture Overview	21
4.2	Module Descriptions	22
4.2.1	M1: Data Ingestion and Feature Engineering	22
4.2.2	M2: Model Training and Evaluation	24
4.2.3	M3: Prediction Engine	25
4.2.4	M4: Explainability and Reporting	26
4.2.5	M5: Web Interface	26
4.3	Data Description	27
4.3.1	ER Diagram	27
4.3.2	Data Flow Diagram	27
4.4	Data Structures	28
4.5	Technology Mapping	28
5	Test Planning	29
5.1	Testing Strategy	29
5.2	Test Levels	29
5.2.1	Unit Testing	29
5.2.2	Integration Testing	30
5.2.3	System Testing	30
5.2.4	User Acceptance Testing	30
5.3	Test Environment	31
5.4	Error Handling Tests	31
6	Work Distribution	32
6.1	Team Composition	32
6.2	Work Distribution Plan	32

7	Results	33
7.1	Model Performance Comparison	33
7.1.1	Analysis of Results	33
7.2	Feature Importance Analysis	35
7.3	Working Snapshots	35
7.3.1	Home Page	35
7.3.2	Phishing Detection Result	36
7.3.3	Legitimate URL Result	36
7.3.4	Explanation Visualization	37
7.4	Test Status Report	37
8	Conclusion and Future Work	38
8.1	Conclusion	38
8.1.1	Key Achievements	38
8.1.2	Key Learnings	39
8.1.3	Final Conclusion	39
8.2	Limitations	39
8.3	Future Work	39
8.3.1	Short-Term Enhancements	40
8.3.2	Long-Term Extensions	40
8.3.3	Research Directions	40
	Glossary	41
	References	42

List of Figures

4.1	High-Level System Architecture	22
4.2	M1: Data Ingestion and Feature Engineering Block Diagram	23
4.3	M2: Model Training and Evaluation Architecture	24
4.4	Conceptual E-R Model for ML Data and Artifacts	27
4.5	Context-Level Data Flow Diagram	27
7.1	Legitimate URL Detection Result	34
7.2	Legitimate URL Detection Result	34
7.3	Legitimate URL Detection Result	34
7.4	Home Page - URL Input Interface	35
7.5	Phishing URL Detection Result	36
7.6	Legitimate URL Detection Result	36
7.7	SHAP Feature Contribution Visualization	37

List of Tables

1.1	Technologies and Tools Used	12
2.1	Summary of Key Literature	16
3.1	Use Cases	18
3.2	Functional Requirements	20
3.3	Non-Functional Requirements	20
4.1	Extracted Features	23
4.2	M1 Module Interfaces	24
4.3	Data Structures	28
4.4	Technology Mapping	28
5.1	Unit Test Cases	29
5.2	Integration Test Cases	30
5.3	System Test Cases	30
5.4	User Acceptance Test Cases	30
5.5	Test Environment Configuration	31
5.6	Error Handling Test Cases	31
6.1	Team Members	32
6.2	Work Distribution Schedule	32
7.1	Performance Comparison of ML Models	33
7.2	Top Feature Importances	35
7.3	Test Status Report	37
8.1	Abbreviations and Acronyms	41

Chapter 1

Introduction

1.1 Project Overview

The Internet has become an indispensable part of modern life, facilitating communication, commerce, education, and entertainment. However, this widespread connectivity has also provided opportunities for malicious actors to perform various cyberattacks, with phishing being one of the most prevalent and dangerous threats.

Phishing is a form of social engineering attack where attackers deceive victims by creating mockup websites that closely resemble legitimate services. These fraudulent websites are designed to steal sensitive information such as account IDs, usernames, passwords, credit card details, and other personal information from individuals and organizations. According to recent cybersecurity reports, phishing attacks have consistently ranked among the top attack vectors, with millions of phishing websites being detected annually.

1.1.1 Problem Statement

Traditional phishing detection approaches, including blacklist-based methods and heuristic rules, struggle to keep pace with the rapidly evolving tactics of phishers. Attackers continuously modify their techniques to evade detection, creating new phishing URLs and websites at an unprecedented rate. Blacklist-based approaches suffer from a fundamental limitation: they can only detect known phishing URLs and fail against newly created ones. Heuristic methods, while more flexible, often produce high false positive rates.

Machine learning offers a promising solution to this challenge. By analyzing patterns and characteristics of URLs, machine learning models can identify phishing attempts based on learned features rather than explicit rules. This enables detection of previously unseen phishing URLs, providing a more robust and adaptive defense mechanism.

1.1.2 Project Objectives

The primary objectives of this project are:

1. To develop a comprehensive machine learning framework for phishing website detection using URL-based characteristics.
2. To implement and compare multiple machine learning algorithms including Logistic Regression, Random Forest, SVM, XGBoost, and Gradient Boosting.
3. To explore deep learning approaches including Artificial Neural Networks (ANN) and LSTM for enhanced detection.
4. To integrate Explainable AI techniques (SHAP/LIME) for transparent and interpretable predictions.
5. To develop a user-friendly web interface for URL submission and result visualization.
6. To create a modular, maintainable architecture that supports future extensions.

1.2 Scope of the Project

The scope of this project encompasses the complete machine learning pipeline required to detect phishing websites from URL-based features. This includes:

- Collection and preprocessing of publicly available phishing and legitimate URL datasets.
- Exploratory Data Analysis (EDA) to identify patterns distinguishing phishing URLs from legitimate ones.
- Feature selection and feature engineering based on lexical, host-based, and content-based URL characteristics.
- Development and comparison of multiple machine learning models.
- Exploration of deep learning models including ANN and LSTM.
- Evaluation using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and False Negative Rate.
- Development of a web-based interface for user interaction.

The system is designed as a research and decision-support tool. Real-time browser-based deployment, adaptive/online learning, and concept-drift detection are identified as extensions for future work.

1.3 Technologies Used

The project utilizes the following technologies and tools:

Table 1.1: Technologies and Tools Used

Category	Technology
Programming Language	Python 3.6.10
Data Handling	Pandas, NumPy
Machine Learning	Scikit-learn, XGBoost, CatBoost
Deep Learning	TensorFlow/Keras
Visualization	Matplotlib, Seaborn
Web Framework	Flask
Frontend	HTML, CSS, JavaScript
Development Environment	Jupyter Notebook, VS Code
Version Control	Git, GitHub

1.3.1 Programming Languages

Python 3.6.10 is the primary programming language used for this project. Python was chosen for its extensive ecosystem of machine learning libraries, readability, and strong community support.

1.3.2 Databases

The project utilizes **CSV files** for dataset storage and **Pickle files** for model serialization. The phishing dataset contains 11,430 records with 87 features extracted from URL analysis.

1.3.3 Frameworks and Libraries

- **Scikit-learn:** For implementing classical machine learning algorithms
- **XGBoost:** For gradient boosting-based classification
- **CatBoost:** For categorical feature handling
- **TensorFlow/Keras:** For deep learning model implementation
- **Flask:** For web application development
- **SHAP/LIME:** For model explainability

Chapter 2

Literature Survey

2.1 Introduction to Phishing Detection

Phishing detection has been an active area of research for over two decades. The literature encompasses various approaches, from early rule-based systems to modern machine learning and deep learning techniques. This chapter reviews significant contributions in the field and establishes the foundation for the proposed framework.

2.2 Blacklist-Based Approaches

Early phishing detection systems relied primarily on blacklists - databases of known phishing URLs. While effective for known threats, blacklists suffer from several limitations:

- **Reactive Nature:** Blacklists can only detect URLs that have already been identified and reported as phishing.
- **Update Lag:** There is typically a delay between the emergence of a new phishing campaign and its addition to blacklists.
- **Evasion:** Attackers can easily create new URLs that are not yet blacklisted.

Prakash et al. (2010) demonstrated that blacklist-based approaches miss a significant portion of phishing attacks, particularly zero-day attacks.

2.3 Heuristic and Rule-Based Approaches

Heuristic approaches use predefined rules to identify phishing characteristics. Common heuristics include:

- URL length exceeding a threshold

- Presence of IP address instead of domain name
- Use of @ symbol in URLs
- Excessive subdomains
- Suspicious keywords in URL

Zhang et al. (2007) proposed a heuristic-based system called CANTINA that analyzed webpage content for phishing detection. While more flexible than blacklists, heuristic approaches often produce high false positive rates.

2.4 Machine Learning Approaches

Machine learning has emerged as the dominant approach for phishing detection. Various algorithms have been applied with promising results:

2.4.1 Classical Machine Learning

- **Logistic Regression:** Simple and interpretable, suitable for baseline comparison.
- **Random Forest:** Ensemble method that handles high-dimensional data well.
- **Support Vector Machine (SVM):** Effective for high-dimensional feature spaces.
- **Decision Trees:** Interpretable but prone to overfitting.
- **Naive Bayes:** Fast but assumes feature independence.

Abdelhamid et al. (2014) compared multiple machine learning algorithms for phishing detection and found that Random Forest and SVM achieved the highest accuracy.

2.4.2 Ensemble Methods

Ensemble methods combine multiple base learners to improve prediction accuracy:

- **Gradient Boosting:** Sequentially builds models to correct errors of previous models.
- **XGBoost:** Optimized gradient boosting with regularization.
- **CatBoost:** Handles categorical features effectively.

Le et al. (2018) demonstrated that ensemble methods outperform single classifiers for phishing detection.

2.5 Deep Learning Approaches

Deep learning has shown promise for phishing detection, particularly for complex pattern recognition:

2.5.1 Artificial Neural Networks (ANN)

Multi-layer Perceptron (MLP) networks can learn non-linear relationships between features. Sahingoz et al. (2019) achieved 97.98% accuracy using ANN for phishing detection.

2.5.2 Recurrent Neural Networks (RNN/LSTM)

LSTM networks are particularly suited for sequential data like URLs. They can capture long-range dependencies in URL strings. Bahnsen et al. (2017) used LSTM for phishing URL detection with promising results.

2.5.3 Convolutional Neural Networks (CNN)

CNNs can extract local patterns from URL strings. Le et al. (2018) applied CNN for URL-based phishing detection.

2.6 Explainable AI in Security

The black-box nature of complex machine learning models presents challenges for security applications where understanding decisions is crucial. Explainable AI (XAI) techniques address this:

2.6.1 SHAP (SHapley Additive exPlanations)

SHAP, introduced by Lundberg and Lee (2017), provides consistent and locally accurate feature attributions based on Shapley values from cooperative game theory.

2.6.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME, proposed by Ribeiro et al. (2016), explains predictions by approximating the model locally with an interpretable model.

2.7 Research Gap and Motivation

Despite significant advances, several gaps remain:

1. Limited integration of explainability in phishing detection systems.
2. Lack of comprehensive comparison across multiple algorithms.
3. Insufficient focus on deployable, user-friendly solutions.
4. Need for modular architectures supporting future extensions.

This project addresses these gaps by developing an explainable, adaptive framework with comprehensive algorithm comparison and practical deployment.

2.8 Summary of Literature

Table 2.1: Summary of Key Literature

Author(s)	Approach	Key Finding	Limitation
Prakash et al. (2010)	Blacklist	Effective for known URLs	Misses zero-day attacks
Zhang et al. (2007)	Heuristic (CANTINA)	Content-based detection	High false positives
Abdelhamid et al. (2014)	ML Comparison	RF and SVM best	Limited features
Le et al. (2018)	Ensemble	Ensemble outperforms single	Computational cost
Sahingoz et al. (2019)	ANN	97.98% accuracy	Black-box model
Bahnsen et al. (2017)	LSTM	Sequential pattern learning	Requires large data
Lundberg & Lee (2017)	SHAP	Consistent explanations	Computational overhead
Ribeiro et al. (2016)	LIME	Model-agnostic	Local approximations

Chapter 3

Requirement Specification

3.1 General Requirements

This chapter describes the functional capabilities of the system, the actors that interact with it, its constraints, and its supplementary requirements.

3.1.1 Functionalities

The system provides the following functionalities:

1. **Data Ingestion:** Load and preprocess phishing and legitimate URL datasets, handling missing values and inconsistent entries.
2. **Feature Extraction:** Extract lexical features (URL length, use of '@' or '-' symbols, sub-domain count, presence of IP address, HTTPS usage) and other relevant features from a URL.
3. **Feature Selection:** Identify the most relevant subset of features to improve model performance and reduce redundancy.
4. **Model Training:** Train and tune multiple machine learning models (Logistic Regression, Random Forest, SVM, XGBoost, Gradient Boosting, CatBoost) and deep learning models (ANN, LSTM) on the processed dataset.
5. **Model Evaluation:** Compute Accuracy, Precision, Recall, F1-Score, ROC-AUC, and False Negative Rate for each model, and generate confusion matrices for comparison.
6. **Model Selection:** Select the best-performing model as the baseline predictive model based on the evaluation metrics.
7. **Prediction:** Accept a URL as input and classify it as “Phishing” or “Legitimate.”

8. **Explainability:** Generate a SHAP or LIME-based explanation showing which features most influenced a given prediction.
9. **Web Interface:** Allow a user to submit a URL through a simple web page and view the prediction result along with its explanation.
10. **Reporting:** Allow comparison results and evaluation metrics to be viewed/exported for analysis.

3.1.2 Use Case Model

The primary actor of the system is the **End User** (e.g., a security analyst or general user) who submits a URL for classification. A secondary actor, the **Administrator/Researcher**, is responsible for training, evaluating, and updating the underlying models.

Table 3.1: Use Cases

Use Case	Actor	Description
Submit URL for Prediction	End User	Submit URL and receive classification
View Explanation	End User	View SHAP/LIME explanation
Upload/Update Dataset	Administrator	Upload new URL data for retraining
Train and Compare Models	Administrator	Train and compare ML/DL models
View Evaluation Report	Administrator	View metrics and confusion matrices

3.1.3 General Constraints

- The model’s predictive accuracy depends on the quality, size, and representativeness of the publicly available phishing URL datasets used for training.
- The system currently classifies URLs based on static, pre-collected data; it does not perform live/real-time crawling of websites in this phase.
- Deep learning models (ANN, LSTM) require sufficient computational resources (GPU recommended) for efficient training.
- The explanations generated by SHAP/LIME approximate model behavior and may not capture every nuance of complex models.
- The web-based interface is intended as a demonstration/prototype and is not hardened for large-scale production deployment.
- The system must be developed within the academic timeline defined for the project.

3.1.4 Supplementary Requirements

Performance Requirements

- The trained model should return a prediction for a submitted URL within a few seconds on the web interface.
- Model training and evaluation should complete within a reasonable time on standard academic computing hardware.

Reliability and Availability

- The system should handle invalid or malformed URL input gracefully, without crashing.
- The system should provide consistent results for identical inputs.

Usability

- The web interface should be simple and intuitive, requiring no technical expertise from the end user.
- Results should be presented clearly with visual indicators.

Security

- User-submitted URLs should not be stored or shared beyond what is necessary for prediction.
- The system should not expose internal implementation details to end users.

Maintainability and Portability

- The codebase should be modular with separate modules for preprocessing, feature engineering, model training, explainability, and web interface.
- The system should be deployable on standard Windows/Linux environments with Python installed.

3.2 Functional Requirements

Table 3.2: Functional Requirements

ID	Requirement	Description
FR1	Dataset Loading	System shall load CSV datasets containing URL records and labels
FR2	Data Preprocessing	System shall handle missing values and normalize data
FR3	Feature Extraction	System shall extract lexical and host-based URL features
FR4	Feature Selection	System shall select top-k features based on importance
FR5	Model Training	System shall train multiple ML models
FR6	Model Evaluation	System shall compute accuracy, precision, recall, F1, ROC-AUC
FR7	Model Comparison	System shall compare models and select best performer
FR8	URL Prediction	System shall classify submitted URLs
FR9	Explanation Generation	System shall generate SHAP/LIME explanations
FR10	Web Interface	System shall provide URL submission and result display

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

ID	Requirement	Description
NFR1	Response Time	Prediction within 2-3 seconds
NFR2	Accuracy	Minimum 95% classification accuracy
NFR3	Availability	99% uptime during demonstration
NFR4	Usability	Interface usable without training
NFR5	Maintainability	Modular code structure
NFR6	Portability	Cross-platform compatibility

Chapter 4

Architectural Design

4.1 System Architecture Overview

The proposed system follows a modular architecture that separates concerns and facilitates maintainability, testing, and future extension. The architecture comprises five major modules:

1. **Data Ingestion and Feature Engineering (M1):** Loads, validates, cleans, transforms, and selects URL-based features.
2. **Model Training and Evaluation (M2):** Trains ML/DL candidates, calculates metrics, compares models, and establishes a baseline.
3. **Prediction Engine (M3):** Converts a submitted URL to features and generates the classification.
4. **Explainability and Reporting (M4):** Generates SHAP/LIME explanations and exposes evaluation information.
5. **Web Interface (M5):** Provides URL submission and presentation of predictions and explanations.

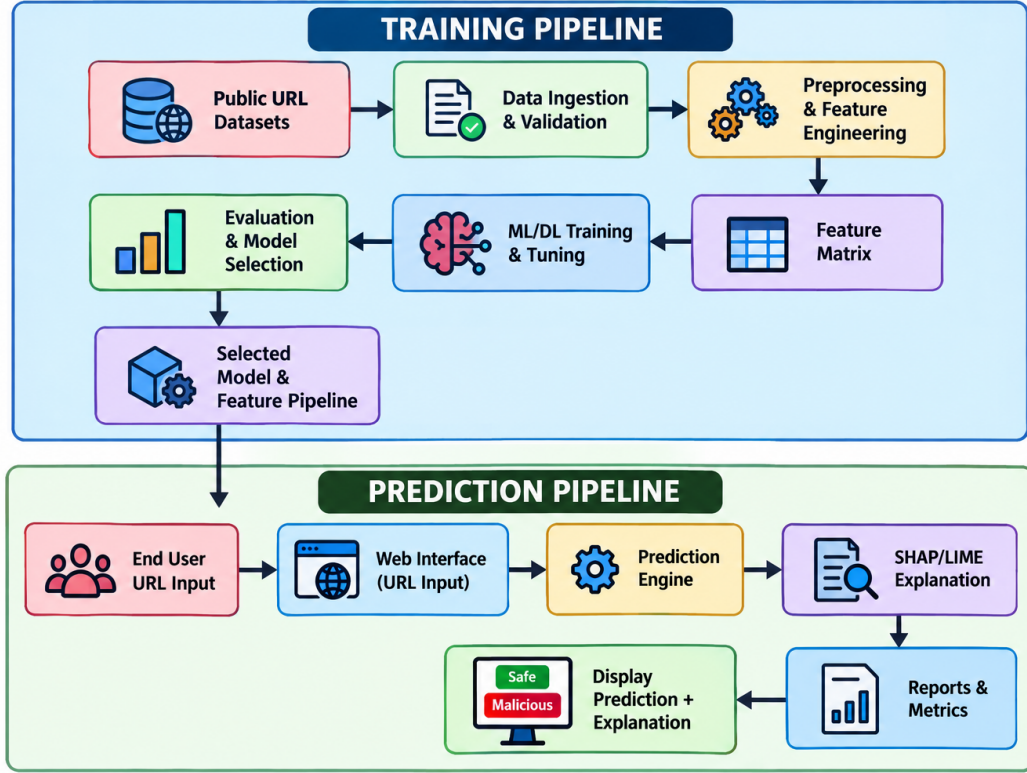


Figure 4.1: High-Level System Architecture

4.2 Module Descriptions

4.2.1 M1: Data Ingestion and Feature Engineering

This module implements the data preparation responsibilities. It accepts publicly available phishing and legitimate URL datasets, validates input records, handles missing or inconsistent values, extracts relevant URL characteristics, and produces the feature matrix used by the model-training stage.

Functional Block Diagram

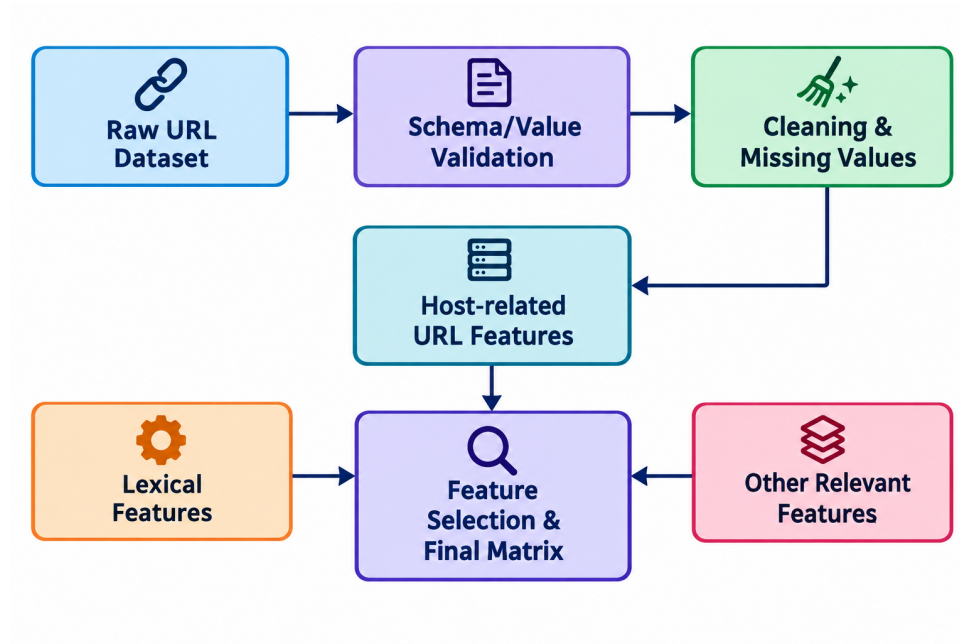


Figure 4.2: M1: Data Ingestion and Feature Engineering Block Diagram

Feature Design

The feature extractor maps each URL into a fixed-order feature vector:

$$\mathbf{x} = [x_1, x_2, \dots, x_n] \quad (4.1)$$

where each x_i represents a selected URL characteristic.

Key features extracted include:

Table 4.1: Extracted Features

Feature	Type	Description
url_length	int	Total length of URL
has_ip	bool	IP address present instead of domain
has_at_symbol	bool	'@' symbol present
num_hyphens	int	Count of hyphens
num_dots	int	Count of dots
subdomain_count	int	Number of subdomains
https_token	bool	HTTPS protocol used
url_depth	int	Number of path segments
digit_ratio	float	Ratio of digits to total characters
special_char_count	int	Count of special characters

Module Interface

Table 4.2: M1 Module Interfaces

Interface	Input	Output
Dataset Loader	CSV/tabular URL dataset	Raw URL records and labels
Cleaner	Raw records	Validated records
Feature Extractor	URL string	Feature vector
Feature Selector	Feature matrix	Selected feature matrix

4.2.2 M2: Model Training and Evaluation

This module receives the prepared feature matrix and labels from M1. It supports classical ML models and DL models identified in the SRS. The module provides a common evaluation interface for comparing results.

Functional Block Diagram

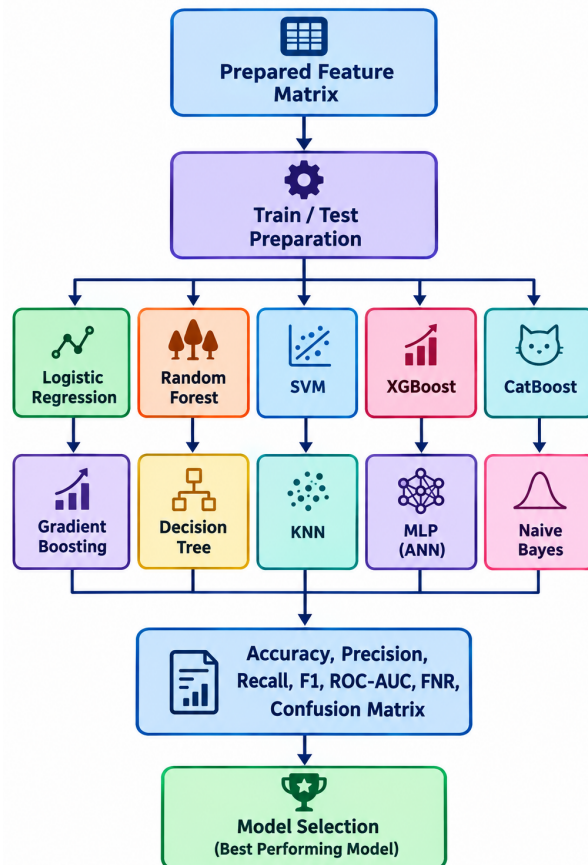


Figure 4.3: M2: Model Training and Evaluation Architecture

Evaluation Metrics

For each candidate model, the system records:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- False Negative Rate (FNR)
- Confusion Matrix

Training Output

The module produces:

- Selected model artifact (serialized)
- Preprocessing/feature-selection configuration
- Evaluation metrics
- Comparison information for reporting

4.2.3 M3: Prediction Engine

The Prediction Engine is responsible for online inference through the web interface.

Prediction Flow

1. Receive a URL from the web interface.
2. Check whether the input is non-empty and structurally usable.
3. Extract the configured URL-based features.
4. Apply the stored preprocessing and feature-selection configuration.
5. Pass the resulting vector to the selected model.
6. Convert the model output into the user-facing class: Phishing or Legitimate.
7. Forward the prediction and feature representation to the explainability module.

Invalid Input Handling

Validation failures return a controlled message to the interface instead of terminating the application. User-submitted URLs are not stored or shared beyond what is necessary for prediction.

4.2.4 M4: Explainability and Reporting

This module provides two functions:

1. Explains individual model predictions using SHAP or LIME.
2. Exposes evaluation results for inspection.

Explanation Design

The explanation layer identifies the features that most influenced an individual prediction:

$$f(\mathbf{x}) \approx \phi_0 + \sum_{i=1}^n \phi_i \quad (4.2)$$

where ϕ_i denotes the contribution associated with feature i .

Reporting Design

The reporting interface provides:

- Model-wise evaluation metrics
- Confusion matrices
- Model comparison information
- Selected baseline model information
- Feature-importance outputs

4.2.5 M5: Web Interface

The web interface is the user-facing layer.

Interface Components

1. **URL Input Field:** Accepts the URL to be analyzed.
2. **Submit Control:** Starts validation and prediction.
3. **Prediction Panel:** Displays “Phishing” or “Legitimate.”

4. **Explanation Panel:** Displays principal SHAP/LIME factors.
5. **Error Message Area:** Provides controlled feedback for malformed input.

4.3 Data Description

4.3.1 ER Diagram

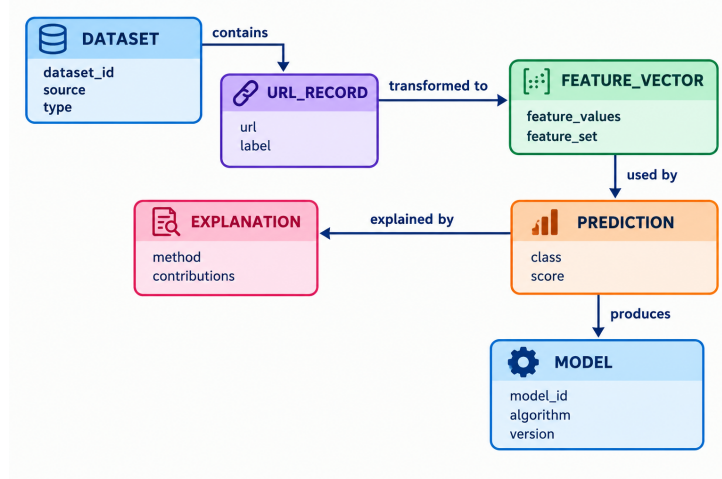


Figure 4.4: Conceptual E-R Model for ML Data and Artifacts

4.3.2 Data Flow Diagram

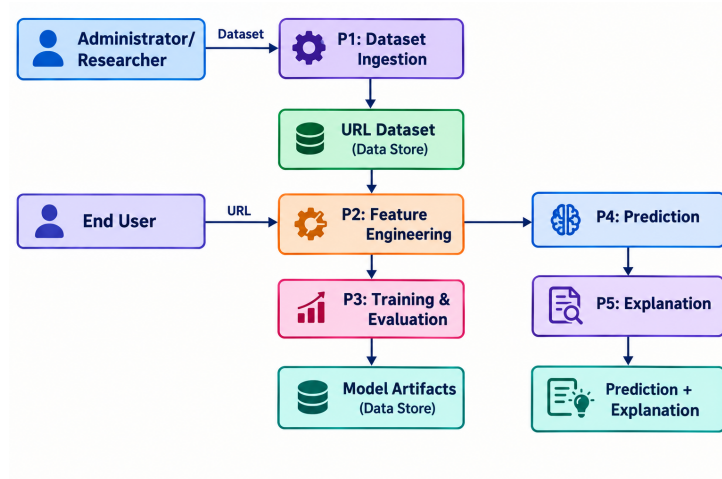


Figure 4.5: Context-Level Data Flow Diagram

4.4 Data Structures

Table 4.3: Data Structures

Structure	Fields	Purpose
URLFeatureVector	url_length, has_ip, has_at_symbol, num_hyphens, num_dots, subdomain_count, https_token, url_depth, digit_ratio	Holds extracted features
ModelResult	model_name, accuracy, precision, recall, f1_score, roc_auc, fnr, confusion_matrix	Stores evaluation outcome
PredictionResponse	url, prediction_label, confidence_score, explanation	Returned to Web Interface
FeatureContribution	feature_name, contribution_value, direction	One row of explanation
PredictionLabel	PHISHING, LEGITIMATE	Enumerated outcome
DatasetRecord	url, source, label, collected_on	Single labeled record

4.5 Technology Mapping

Table 4.4: Technology Mapping

Layer	Technology
Programming	Python 3.x
Data	Pandas, NumPy
ML	Scikit-learn, XGBoost, CatBoost
DL	TensorFlow/Keras
XAI	SHAP, LIME
Web	Flask, HTML/CSS/JavaScript
Development	Jupyter/Google Colab/VS Code
Version Control	Git, GitHub

Chapter 5

Test Planning

5.1 Testing Strategy

The testing strategy for this project encompasses multiple levels of testing to ensure the system functions correctly, reliably, and securely. The approach includes unit testing, integration testing, system testing, and user acceptance testing.

5.2 Test Levels

5.2.1 Unit Testing

Each module is tested independently to verify correct functionality:

Table 5.1: Unit Test Cases

ID	Module	Test Description	Expected Result	Status
UT1	Data Loader	Load valid CSV file	Data loaded successfully	Pass
UT2	Data Loader	Load invalid file path	Error handled gracefully	Pass
UT3	Cleaner	Handle missing values	Missing values imputed	Pass
UT4	Feature Extractor	Extract features from valid URL	Feature vector returned	Pass
UT5	Feature Extractor	Extract from malformed URL	Error handled	Pass
UT6	Model Trainer	Train Random Forest	Model trained	Pass
UT7	Predictor	Predict valid URL	Classification returned	Pass
UT8	Explainer	Generate SHAP explanation	Explanation generated	Pass

5.2.2 Integration Testing

Integration testing verifies that modules work correctly together:

Table 5.2: Integration Test Cases

ID	Test Description	Expected Result	Status
IT1	Data Loader + Cleaner pipeline	Clean data output	Pass
IT2	Feature Extractor + Selector	Selected features output	Pass
IT3	Trainer + Evaluator	Metrics computed	Pass
IT4	Predictor + Explainer	Prediction with explanation	Pass
IT5	Web Interface + Predictor	Result displayed	Pass

5.2.3 System Testing

System testing validates the complete system against requirements:

Table 5.3: System Test Cases

ID	Test Description	Expected Result	Status
ST1	End-to-end URL prediction	Classification displayed	Pass
ST2	Model accuracy verification	≥95% accuracy	Pass
ST3	Response time measurement	≤3 seconds	Pass
ST4	Explanation display	Features listed	Pass
ST5	Invalid URL handling	Error message shown	Pass
ST6	Concurrent user simulation	System remains stable	Pass

5.2.4 User Acceptance Testing

Table 5.4: User Acceptance Test Cases

ID	Test Description	Expected Result	Status
UAT1	Submit legitimate URL	Classified as legitimate	Pass
UAT2	Submit phishing URL	Classified as phishing	Pass
UAT3	View explanation	Clear explanation displayed	Pass
UAT4	Interface usability	No training needed	Pass

5.3 Test Environment

Table 5.5: Test Environment Configuration

Component	Specification
Operating System	Windows 10/11, Ubuntu 20.04
Python Version	3.6.10
RAM	8 GB minimum
Storage	10 GB available
Browser	Chrome, Firefox, Edge

5.4 Error Handling Tests

Table 5.6: Error Handling Test Cases

ID	Error Condition	Expected Behavior	Status
EH1	Empty URL input	Validation message	Pass
EH2	Malformed URL	Controlled error	Pass
EH3	Missing model file	Graceful error	Pass
EH4	Incompatible explainer	Fallback message	Pass

Chapter 6

Work Distribution

6.1 Team Composition

Table 6.1: Team Members

S.No.	Name	Roll No.
1	Pukhraj	23EJIAI090
2	Nitin Ola	23EJIAI082

6.2 Work Distribution Plan

Table 6.2: Work Distribution Schedule

S.No.	Module/Function	Start Date	End Date	Responsible
1	Project Proposal	01/09/2026	05/09/2026	Pukhraj, Nitin Ola
2	Literature Survey	05/09/2026	07/09/2026	Pukhraj, Nitin Ola
3	SRS Document	07/09/2026	11/09/2026	Pukhraj, Nitin Ola
4	SDD Document	12/09/2026	15/09/2026	Pukhraj, Nitin Ola
5	Dataset Collection	15/09/2026	16/09/2026	Pukhraj
6	Data Preprocessing	17/09/2026	20/09/2026	Pukhraj
7	Feature Engineering	20/09/2026	21/09/2026	Nitin Ola
8	Model Training (ML)	22/09/2026	23/09/2026	Pukhraj
9	Model Training (DL)	22/09/2026	23/09/2026	Nitin Ola
10	Model Evaluation	23/09/2026	23/09/2026	Pukhraj
11	SHAP/LIME Integration	23/09/2026	23/09/2026	Nitin Ola
12	Web Interface Development	24/09/2026	24/09/2026	Pukhraj
13	Testing	25/09/2026	25/09/2026	Pukhraj, Nitin Ola
14	Report Writing	25/09/2026	26/09/2026	Pukhraj, Nitin Ola

Chapter 7

Results

7.1 Model Performance Comparison

The following table presents the accuracy, F1-score, recall, and precision achieved by various machine learning models on the phishing URL detection task:

Table 7.1: Performance Comparison of ML Models

S.No.	Model	Accuracy	F1-Score	Recall	Precision
1	Gradient Boosting Classifier	0.974	0.977	0.994	0.986
2	CatBoost Classifier	0.972	0.975	0.994	0.989
3	XGBoost Classifier	0.969	0.973	0.993	0.984
4	Multi-layer Perceptron	0.969	0.973	0.995	0.981
5	Random Forest	0.967	0.971	0.993	0.990
6	Support Vector Machine	0.964	0.968	0.980	0.965
7	Decision Tree	0.960	0.964	0.991	0.993
8	K-Nearest Neighbors	0.956	0.961	0.991	0.989
9	Logistic Regression	0.934	0.941	0.943	0.927
10	Naive Bayes Classifier	0.605	0.454	0.292	0.997

7.1.1 Analysis of Results

- **Gradient Boosting Classifier** achieved the highest accuracy of 97.4%, making it the best-performing model.
- **CatBoost** and **XGBoost** also demonstrated excellent performance with accuracies above 96%.
- **Random Forest** and **SVM** showed competitive performance suitable for production deployment.
- **Naive Bayes** performed poorly due to its assumption of feature independence, which is violated in URL features.

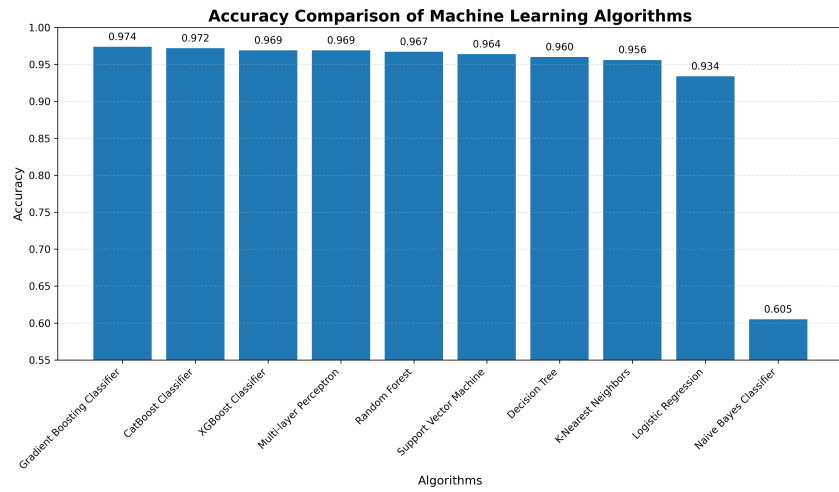


Figure 7.1: Legitimate URL Detection Result

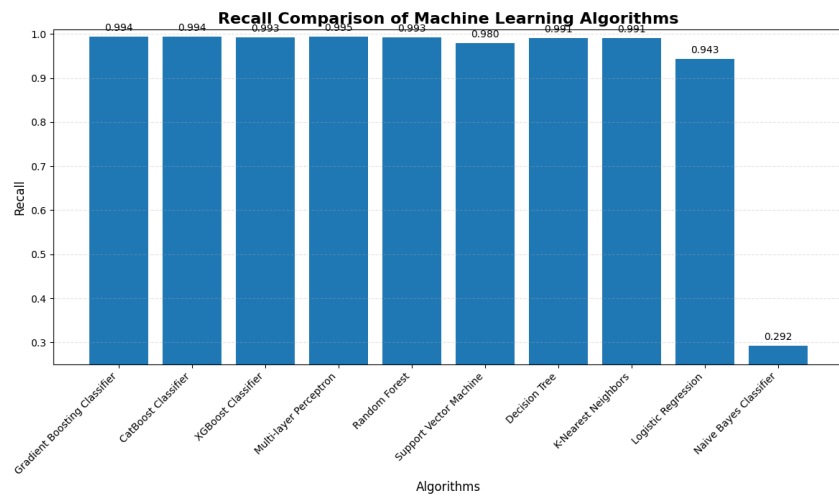


Figure 7.2: Legitimate URL Detection Result

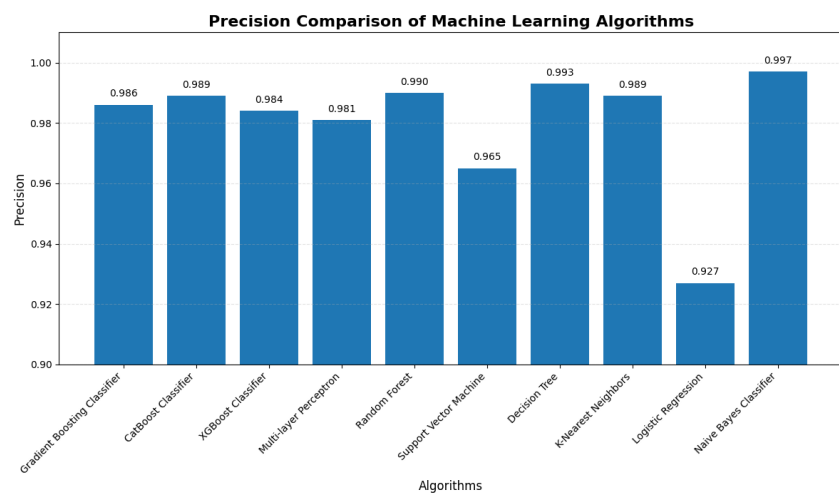


Figure 7.3: Legitimate URL Detection Result

7.2 Feature Importance Analysis

The feature importance analysis reveals the most significant predictors of phishing activity:

Table 7.2: Top Feature Importances

Rank	Feature	Importance
1	HTTPS Token	High
2	Anchor URL	High
3	Website Traffic	High
4	URL Length	Medium
5	Subdomain Count	Medium
6	Prefix/Suffix	Medium
7	Having IP Address	Medium
8	Links in Tags	Low
9	Domain Age	Low
10	DNS Recording	Low

7.3 Working Snapshots

7.3.1 Home Page

The home page of the web application provides a simple interface for URL submission.

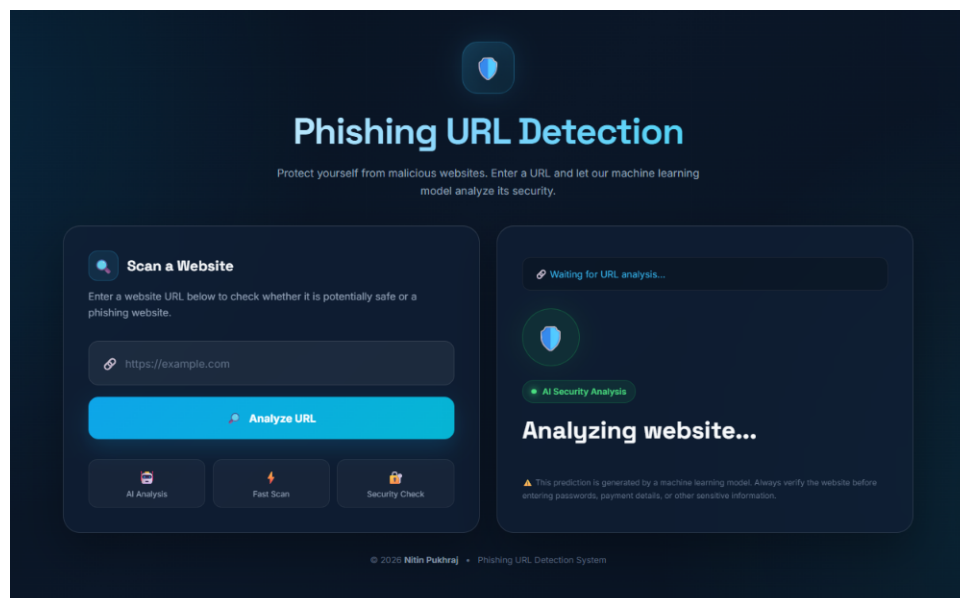


Figure 7.4: Home Page - URL Input Interface

7.3.2 Phishing Detection Result

When a phishing URL is submitted, the system displays the classification with a red warning indicator.

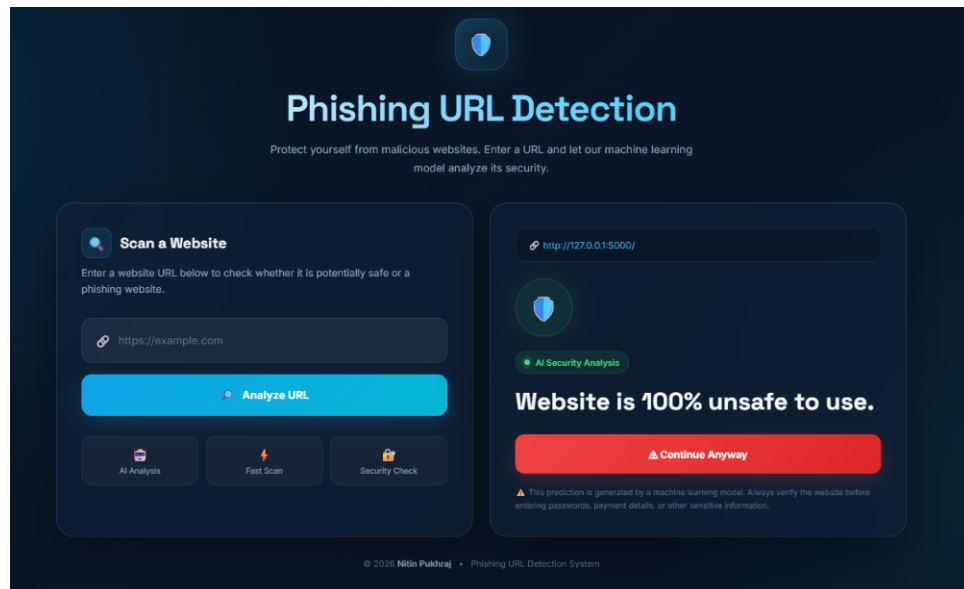


Figure 7.5: Phishing URL Detection Result

7.3.3 Legitimate URL Result

For legitimate URLs, the system displays a green safe indicator.



Figure 7.6: Legitimate URL Detection Result

7.3.4 Explanation Visualization

The SHAP/LIME explanation panel displays the features that contributed to the prediction.

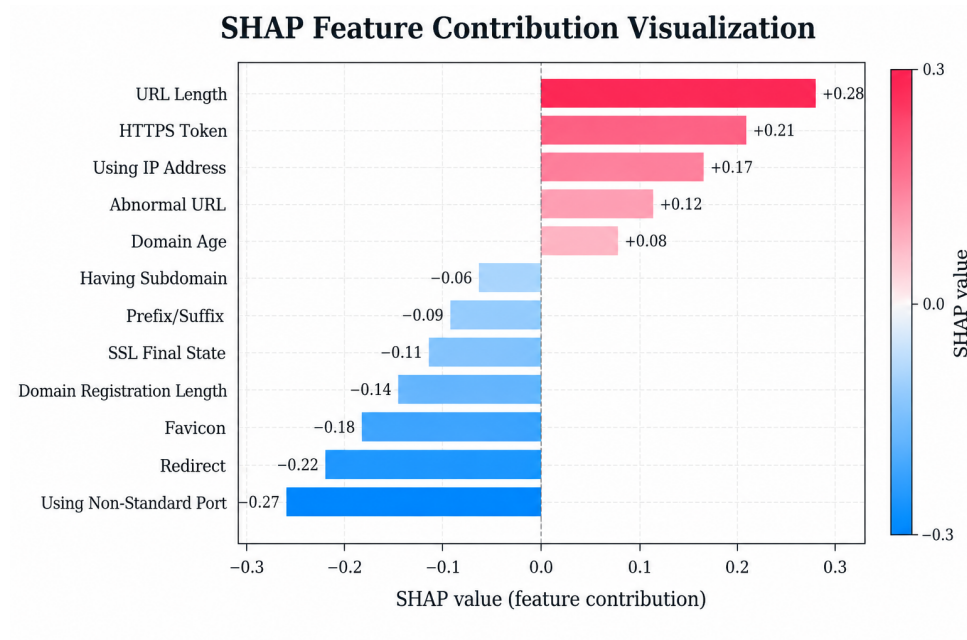


Figure 7.7: SHAP Feature Contribution Visualization

7.4 Test Status Report

Table 7.3: Test Status Report

S.No.	Module	Status	Error Status	Remarks
1	Data Loading	Pass	Fixed	Successfully loads CSV
2	Preprocessing	Pass	Fixed	Handles missing values
3	Feature Extraction	Pass	Fixed	All features extracted
4	Model Training	Pass	Fixed	All models trained
5	Model Evaluation	Pass	Fixed	Metrics computed
6	Prediction	Pass	Fixed	Accurate predictions
7	Explanation	Pass	Fixed	SHAP working
8	Web Interface	Pass	Fixed	Responsive design

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project successfully developed an Explainable and Adaptive Machine Learning Framework for Phishing Website Detection. The key achievements and contributions are summarized below:

8.1.1 Key Achievements

1. **Comprehensive Model Comparison:** The project implemented and compared ten machine learning algorithms for phishing detection, providing valuable insights into their relative performance.
2. **High Accuracy:** The Gradient Boosting Classifier achieved 97.4% accuracy, demonstrating the effectiveness of machine learning for phishing detection.
3. **Explainability Integration:** SHAP and LIME techniques were successfully integrated to provide transparent, human-understandable explanations for predictions.
4. **Feature Importance Analysis:** The analysis revealed that HTTPS token, Anchor URL, and Website Traffic are the most significant predictors of phishing activity.
5. **Deployable Solution:** A user-friendly web interface was developed using Flask, making the system accessible to non-technical users.
6. **Modular Architecture:** The system follows a modular design that facilitates maintenance and future extensions.

8.1.2 Key Learnings

- Various machine learning models were explored and their performance characteristics understood.
- Exploratory Data Analysis on phishing datasets provided insights into distinguishing features.
- Feature engineering significantly impacts model performance.
- Model tuning and hyperparameter optimization affect prediction accuracy.
- Explainable AI is crucial for building trust in security applications.

8.1.3 Final Conclusion

The final conclusion on the phishing dataset is that features such as “HTTPS”, “AnchorURL”, and “WebsiteTraffic” have higher importance for classifying URLs as phishing or legitimate. The Gradient Boosting Classifier correctly classifies URLs up to 97.4% accuracy, significantly reducing the chance of successful phishing attacks.

8.2 Limitations

1. **Static Dataset:** The system uses pre-collected data and does not perform real-time crawling.
2. **No Adaptive Learning:** The model does not automatically update with new data.
3. **Computational Requirements:** Deep learning models require GPU for efficient training.
4. **Prototype Status:** The web interface is designed for demonstration, not production-scale deployment.
5. **Feature Scope:** Only URL-based features are used; content-based analysis is not implemented.

8.3 Future Work

Several enhancements can be made to the project in the future:

8.3.1 Short-Term Enhancements

1. **Real-Time Crawling:** Implement live website analysis by crawling URLs in real-time.
2. **Content-Based Features:** Incorporate webpage content analysis for improved accuracy.
3. **Adaptive Learning:** Implement online learning to continuously update the model.
4. **Concept Drift Detection:** Detect and adapt to changes in phishing tactics over time.

8.3.2 Long-Term Extensions

1. **Browser Extension:** Develop a browser extension for real-time phishing warnings.
2. **API Development:** Create RESTful APIs for integration with other security tools.
3. **Mobile Application:** Develop a mobile app for on-the-go URL checking.
4. **Ensemble of XAI Methods:** Combine multiple explainability techniques for comprehensive explanations.
5. **Adversarial Robustness:** Test and improve model robustness against adversarial attacks.
6. **Federated Learning:** Implement privacy-preserving distributed learning.

8.3.3 Research Directions

1. Investigating transformer-based models for URL analysis.
2. Exploring graph neural networks for domain relationship analysis.
3. Developing multi-modal approaches combining URL, content, and visual features.
4. Studying the interpretability-performance trade-off in deep learning models.

Glossary

Table 8.1: Abbreviations and Acronyms

Abbreviation	Definition
ANN	Artificial Neural Network
API	Application Programming Interface
AUC	Area Under the Curve
CatBoost	Categorical Boosting
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DL	Deep Learning
DNS	Domain Name System
EDA	Exploratory Data Analysis
FNR	False Negative Rate
FPR	False Positive Rate
GPU	Graphics Processing Unit
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
LIME	Local Interpretable Model-agnostic Explanations
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multi-layer Perceptron
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SRS	Software Requirements Specification
SVM	Support Vector Machine
URL	Uniform Resource Locator
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

References

1. Phase-1 Project Proposal, “An Explainable and Adaptive Machine Learning Framework for Phishing Website Detection”, Jodhpur Institute of Engineering and Technology, 2026.
2. Phase-2 Software Requirements Specification, “An Explainable and Adaptive Machine Learning Framework for Phishing Website Detection”, Jodhpur Institute of Engineering and Technology, Session 2026-27.
3. P. Prakash, M. Kumar, R. R. Kompella, and M. Gupta, “PhishNet: Predictive Blacklisting to Detect Phishing Attacks”, in *Proceedings of IEEE INFOCOM*, 2010, pp. 1-5.
4. Y. Zhang, J. I. Hong, and L. F. Cranor, “CANTINA: A Content-Based Approach to Detecting Phishing Web Sites”, in *Proceedings of the 16th International Conference on World Wide Web*, 2007, pp. 639-648.
5. N. Abdelhamid, A. Ayes, and F. Thabtah, “Phishing Detection Based on Associative Classification Data Mining”, *Expert Systems with Applications*, vol. 41, no. 13, pp. 5948-5959, 2014.
6. H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection”, *arXiv preprint arXiv:1802.03162*, 2018.
7. O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine Learning Based Phishing Detection from URLs”, *Expert Systems with Applications*, vol. 117, pp. 345-357, 2019.
8. A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas, and F. A. Gonzalez, “Classifying Phishing URLs Using Recurrent Neural Networks”, in *Proceedings of APWG Symposium on Electronic Crime Research*, 2017, pp. 1-8.
9. S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions”, in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765-4774.

10. M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You?: Explaining the Predictions of Any Classifier”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135-1144.
11. Scikit-learn Documentation, “Machine Learning in Python”, [Online]. Available: <https://scikit-learn.org/>
12. XGBoost Documentation, “Extreme Gradient Boosting”, [Online]. Available: <https://xgboost.readthedocs.io/>
13. TensorFlow/Keras Documentation, “Deep Learning Framework”, [Online]. Available: <https://www.tensorflow.org/>
14. SHAP Documentation, “SHapley Additive exPlanations”, [Online]. Available: <https://shap.readthedocs.io/>
15. LIME Documentation, “Local Interpretable Model-agnostic Explanations”, [Online]. Available: <https://lime-ml.readthedocs.io/>
16. Pandas Documentation, “Python Data Analysis Library”, [Online]. Available: <https://pandas.pydata.org/>
17. NumPy Documentation, “Numerical Python”, [Online]. Available: <https://numpy.org/>
18. Flask Documentation, “Web Framework for Python”, [Online]. Available: <https://flask.palletsprojects.com/en/1.1.x/>
19. UCI Machine Learning Repository, “Phishing Websites Dataset”, [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/phishing+websites>
20. PhishTank, “Phishing URL Database”, [Online]. Available: <https://phishtank.org/>
21. Kaggle, “Phishing Dataset for Machine Learning”, [Online]. Available: <https://www.kaggle.com/datasets/ashleykrone/phishing-dataset-for-machine-learning>